MXB362

Computer Lab Portfolio

Problem Solving Task 1 - Weeks 2 to 6

Student	Owen Ferguson
Student number	n11595744

Portfolio contents

Week 2 - Introduction to Visualisation in MATLAB, Part 1

Task 1: Mauna Loa CO2 graph
Task 2: GBRMPA vector map
Task 3: 3D stack of squares

Week 3 - Introduction to Visualisation in MATLAB, Part 2

Task 1: Methamphetamine laboratory maps
Task 2: Logistic map and bifurcation diagram
Task 3: Mandelbrot set and zooms

Week 5 - Animation in MATLAB

Task 1: London Underground population animation
Task 2: Pendulum animation

Week 6 - Volume Visualisation in R

Task 1: MRI cylindrical cutout
Task 2: Tooth slice
Task 3: Tooth transfer function render

Mauna Loa CO2 Graph

Create and export a TIFF showing the Mauna Loa atmospheric CO2 visualisation in a two-panel layout.

Code - week2/co2_mauna_loa.m

Source file: week2/co2_mauna_loa.m

```
load('ML_CO2_data.mat')
figure(1)
clf
hold on
box on
pp = plot(Data(:,3),Data(:,4))

get(pp)
set(pp,'linewidth',2,'color',[1 0 0])
set(pp,'linestyle',':')

xlabel('Year','fontsize',15)
ylabel('Atmospheric CO_2 concentration (ppm)','fontsize',15)
title('Atmospheric CO_2 concentration at Mauna Loa,','fontsize',16)
L = legend('Mauna Loa monthly averages');
set(L,'fontsize',14,'location','northwest')

xlim([1958 2020])
ylim([310 415])

% Second plot
subplot('position', [0.1 0.3 0.85 0.65])
hold on
box on
pp = plot(Data(:,3),Data(:,4));

xlabel('Year','fontsize',15)
ylabel('Atmospheric CO_2 concentration (ppm)','fontsize',15)
title('Atmospheric Carbon dioxide concentration at Mauna Loa, Hawai''i','fontsize',16)
L = legend('Mauna Loa monthly averages')
set(L,'fontsize',14,'location','northwest')
xlim([1958 2020])
ylim([310 415])

% export to tiff
set(gcf,'paperunits','centimeters','paperposition',[0 0 21 30])
print('-dtiff','-r300','Mauna_loa.tiff')
```

Create and export a TIFF showing the Mauna Loa atmospheric CO2 visualisation in a two-panel layout.

Execution evidence

```
>> co2_mauna_loa

pp =

  Line with properties:

            Color: [0.1490 0.5490 0.8660]
        LineStyle: '-'
        LineWidth: 0.5000
           Marker: 'none'
       MarkerSize: 6
  MarkerFaceColor: 'none'
            XData: [1.9582e+03 1.9583e+03 1.9584e+03 1.9585e+03 1.9586e+03 1.9587e+03 1.9589e+03 1.9590e+03 … ] (1×729 double)
            YData: [315.7100 317.4500 317.5000 315.8600 314.9300 313.2000 313.3300 314.6700 315.6200 … ] (1×729 double)

  Show all properties

    AffectAutoLimits: on
   AlignVertexCenters: off
          Annotation: [1×1 matlab.graphics.eventdata.Annotation]
        BeingDeleted: off
          BusyAction: 'queue'
       ButtonDownFcn: ''
            Children: [0×0 GraphicsPlaceholder]
            Clipping: on
               Color: [0.1490 0.5490 0.8660]
           ColorMode: 'auto'
         ContextMenu: [0×0 GraphicsPlaceholder]
           CreateFcn: ''
     DataTipTemplate: [1×1 matlab.graphics.datatip.DataTipTemplate]
           DeleteFcn: ''
         DisplayName: ''
    HandleVisibility: 'on'
             HitTest: on
       Interruptible: on
            LineJoin: 'round'
           LineStyle: '-'
       LineStyleMode: 'auto'
           LineWidth: 0.5000
              Marker: 'none'
     MarkerEdgeColor: 'auto'
     MarkerFaceColor: 'none'
       MarkerIndices: [1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 … ] (1×729 uint64)
          MarkerMode: 'auto'
          MarkerSize: 6
              Parent: [1×1 Axes]
       PickableParts: 'visible'
            Selected: off
   SelectionHighlight: on
         SeriesIndex: 1
         SourceTable: [0×0 table]
                 Tag: ''
                Type: 'line'
            UserData: []
             Visible: on
               XData: [1.9582e+03 1.9583e+03 1.9584e+03 1.9585e+03 1.9586e+03 1.9587e+03 1.9589e+03 … ] (1×729 double)
           XDataMode: 'manual'
         XDataSource: ''
           XVariable: ''
               YData: [315.7100 317.4500 317.5000 315.8600 314.9300 313.2000 313.3300 314.6700 315.6200 … ] (1×729 double)
           YDataMode: 'manual'
         YDataSource: ''
           YVariable: ''
               ZData: [1×0 double]
           ZDataMode: 'auto'
         ZDataSource: ''
           ZVariable: ''


L =

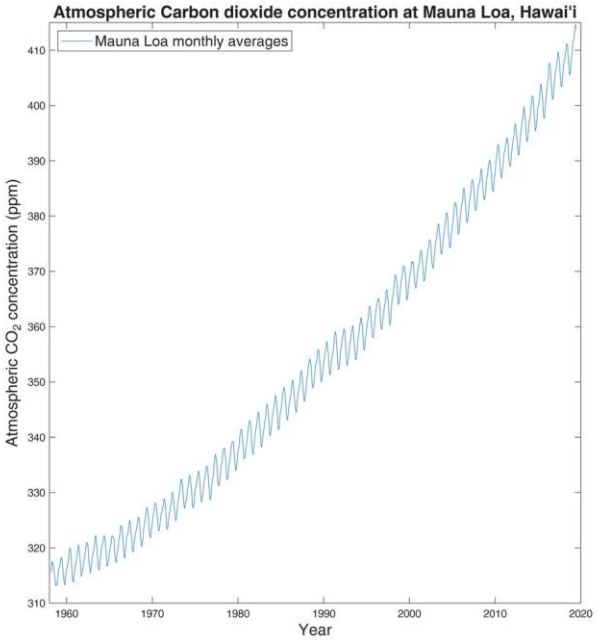
  Legend (Mauna Loa monthly averages) with properties:

       String: {'Mauna Loa monthly averages'}
     Location: 'northeast'
   Orientation: 'vertical'
      FontSize: 9
      Position: [0.7443 0.9062 0.1955 0.0278]
         Units: 'normalized'

  Show all properties

>>
```

Output



Mauna_loa.tiff - display preview

GBRMPA Vector Map

Plot the Great Barrier Reef Marine Park and green zones, then export the map as a vector graphics file.

Code - week2/greenzonew1p2.m

Source file: week2/greenzonew1p2.m

```
clear, close all

figure(2), clf, hold on, box on

load AustOutline_MXB362.mat
pt = patch(Outline(:,1),Outline(:,2),[0.8 0.8 0.8]);
set(pt,'facecolor',[1 0.9 0.65])

load ReefOutline_MXB362
plot(ReefRaw(:,1),ReefRaw(:,2),'b')

load GreenZones_MXB362
NumZones = length(GreenOutline);
for a = 1:NumZones
    pt = patch(GreenOutline{a}(1,:),GreenOutline{a}(2,:),'g'); set(pt,'edgecolor','none','facealpha',0.5)
end

xlim([142.3 154.5])
ylim([-26 -9])
GBRMPA_OUTLINE = [142.5 -10.7; 145.0 -10.7; 145.0 -13.0; 147.0 -17.5; 153.0 -21.0; 154.0 -24.5; 152.0 -24.5];
plot(GBRMPA_OUTLINE(:,1), GBRMPA_OUTLINE(:,2), 'r-', 'linewidth', 1.5)

set(gca,'color',[0.94 1 1])
text(145,-21,'Queensland','fontsize',17)
T = text(149.8,-18.74,'GBRMP boundary','fontsize',12);
set(T,'fontangle','italic','color','r')
set(T,'rotation',-18)

axes('position',[0.15 0.12 0.2 0.1])
axis equal; hold on
patch(Outline(:,1),Outline(:,2),[1 1 1]);
pt = patch([142.3 154.5 154.5 142.3],[-26 -26 -9 -9],[0 0 0]);
set(pt,'facealpha',0.2)
axis off

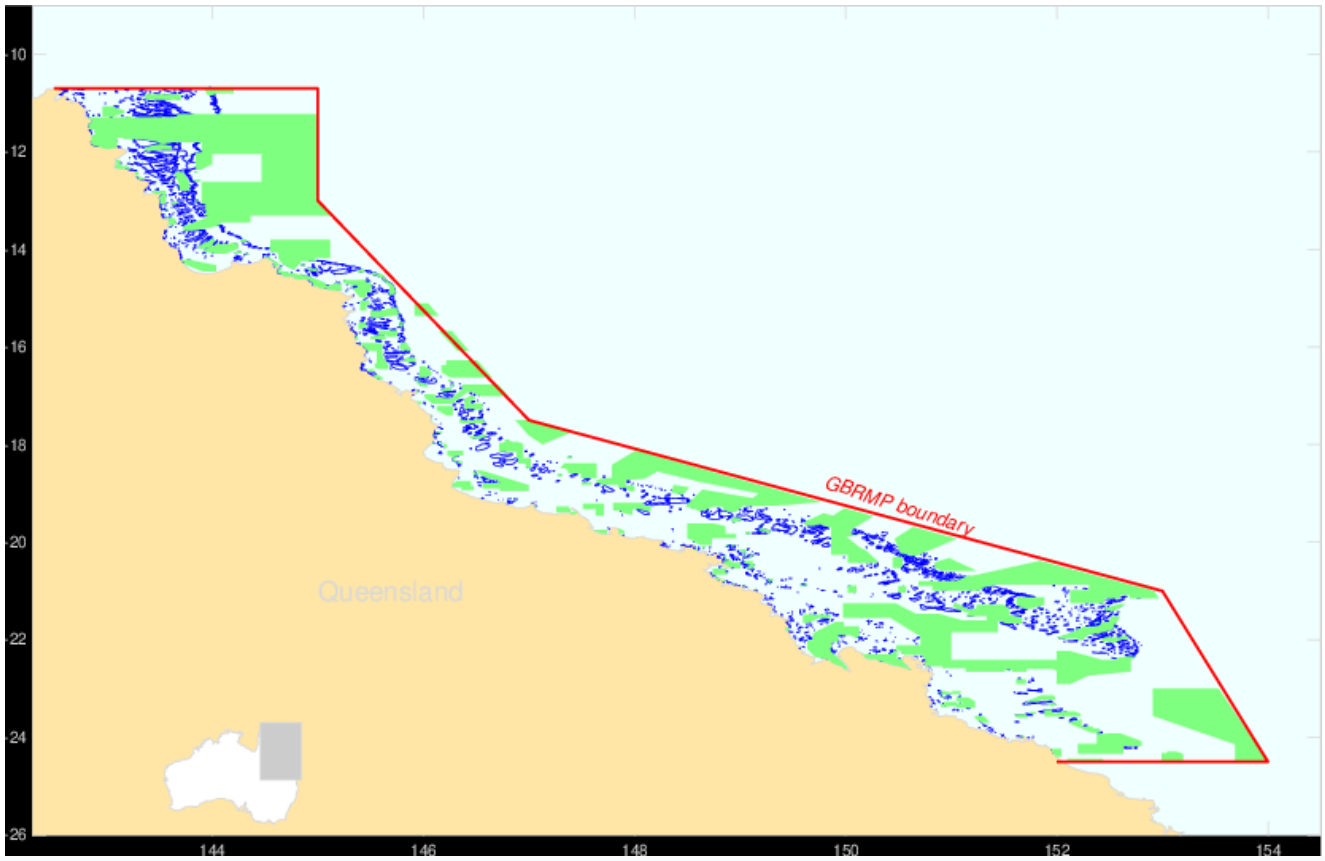
print('-depsc','-r300','-painters','GBRMPA_plot.eps')   % or -dsvg / -dpdf
```

Plot the Great Barrier Reef Marine Park and green zones, then export the map as a vector graphics file.

Execution evidence

```
>> greenzonew1p2
>>
```

Output



GBRMPA_plot.eps - rasterised display preview

3D Stack of Squares

Create a 3D stack with twice the original number of squares and export it as a JPEG image.

Code - week2/week1t3.m

Source file: week2/week1t3.m

```
figure(3)
clf
hold on
box on

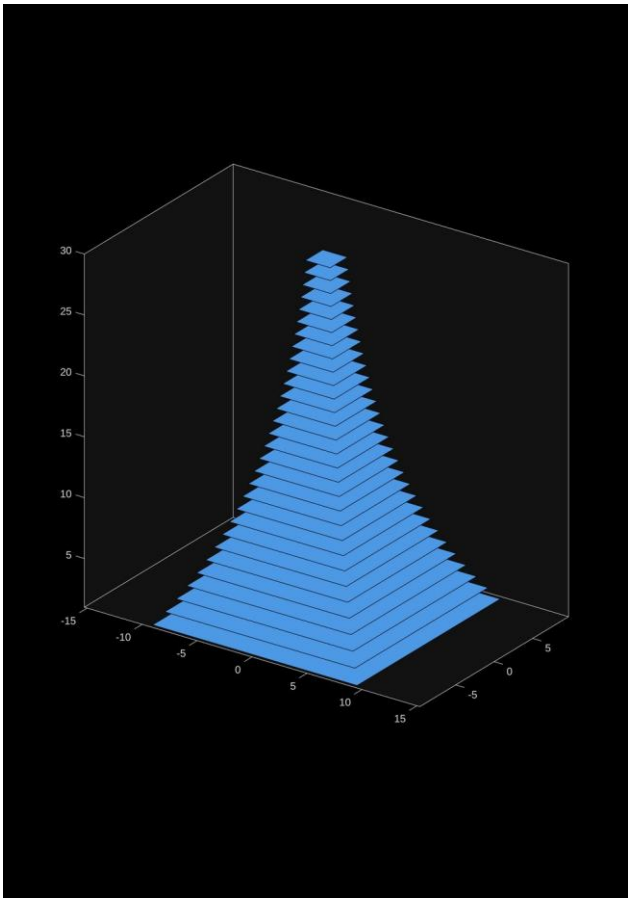
N = 30;
for i = 1:N
      s = 10 * 0.93^i;                % diminishing square half-width
patch([-s s s -s],[-s -s s s],[i i i i], [0.3 0.6 0.9], 'EdgeColor','k')
end
axis equal
view(35,25)                          % a nice 3-D angle for the still image

% export as JPG
set(gcf,'paperunits','centimeters','paperposition',[0 0 21 30])
print('-djpeg','-r300','stack_of_squares.jpg')
```

Execution evidence

```
>> week1t3
>>
```

Output



stack_of_squares.jpg

Methamphetamine Laboratory Maps

Produce a county map with laboratory locations and a state map colour-coded by relative laboratory density.

Code - week3/Methlabscript.m

Source file: week3/Methlabscript.m

```
clear all

load USA_County_struct.mat
load USA_State_struct.mat

[D,T] = xlsread('Geocoded_MethLabs.xlsx');
Loc = [D(:,11) D(:,10)];

figure(1); clf, hold on; box on;

for s = 1:length(S)
   XX = S(s).X;
   YY = S(s).Y;

   plot(XX,YY,'color',0.8.*ones(1,3));
end

ml = plot(Loc(:,1),Loc(:,2),'r.','markersize',4);

FS = 16;
title('Locations of methamphetamine labortories','fontsize',FS);
text(-124,24.3,'Source: DEA National Clandestine Laboratory Register','fontsize',FS)

set(gca,'xtick',[],'ytick',[])
xlim([-126 -66]);
ylim([23.3 50]);


figure(2), clf, hold on; box on;

CL = hot(3200);
CL = CL(end:-1:1,:);

for s = 1:length(S_states)
   XX = S_states(s).X;
   YY = S_states(s).Y;

   LabsInThisStates(s) = sum(inpolygon(Loc(:,1),Loc(:,2),XX,YY));

   ThisColor = ceil(LabsInThisStates(s))+1;

   P = nanpatch_MXB362 (XX,YY,CL(ThisColor,:));
end

FS = 16;
set(gca,'xtick',[],'ytick',[]);
xlim([-126, -66]); ylim([23.5 50])
colormap(CL); C = colorbar;
title('Number of methamphetamine labs in each state','fontsize',FS);
set(C,'Ticks',linspace(0,1,9),'ticklabels',linspace(0,3200,9),'fontsize',FS-2);
```

Code - nanpatch_MXB362.m (helper function)

Source file: week3/nanpatch_MXB362.m

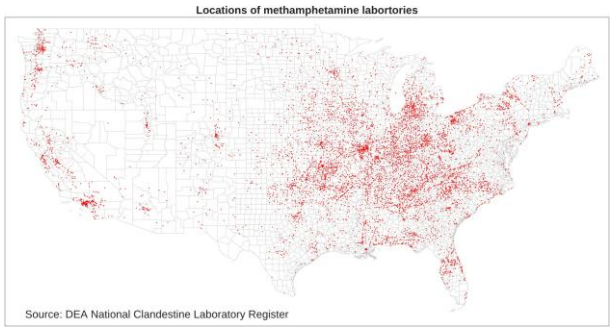
```
001  function P = nanpatch_MXB362(XX,YY,CL)
002
003  F = [0 find(isnan(XX))];
004  for f = 1:length(F)-1
005      XXX = XX(F(f)+1:F(f+1)-1);
006      YYY = YY(F(f)+1:F(f+1)-1);
007
008      P = patch(XXX,YYY,CL);
009      set(P,'edgecolor',0.8.*ones(1,3));
010  end
```

Produce a county map with laboratory locations and a state map colour-coded by relative laboratory density.

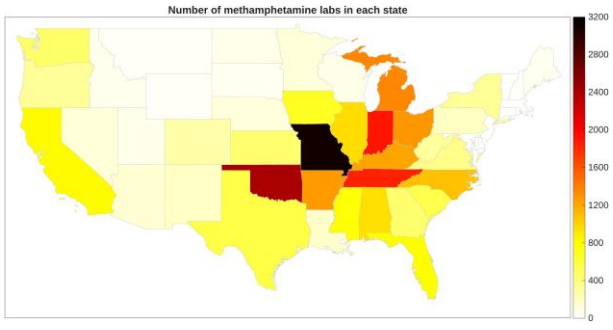
Execution evidence

```
>> Methlabscript
>>
```

Output



Task1_Figure_1.png - laboratory locations



Task1_Figure_2.png - state density map

Logistic map and bifurcation diagram

Show logistic-map time series and the bifurcation diagram for growth-rate values spanning 2.6 to 3.8.

Code - week3/task2.m

Source file: week3/task2.m

```
clear all

r = 2.2;
N = 0.3;

figure(1), clf

subplot(3,1,1), box on

for t = 1:99
   N(t+1) = r*N(t)*(1 - N(t));
end

plot([1:100],N,'linewidth',2)
xlim([1 100]); ylim([0 1.1])
title(['Growth rate  = ' num2str(r,3)])

%for different values of r

r = 3.2;
subplot(3,1,2), box on
for t = 1:100
   N(t+1) = r*N(t)*(1 - N(t));
end
plot(N,'linewidth',2)
xlim([1 100]); ylim([0 1.1])
title(['Growth Rate = ' num2str(r,3)])

%repeat with another different r

r = 4;
subplot(3,1,3), box on
for t = 1:100
   N(t+1) = r*N(t)*(1 - N(t));
end
plot(N,'linewidth',2)
xlim([1 100]); ylim([0 1.1])
title(['Growth Rate = ' num2str(r,3)])

% Initialise new figure
figure(2), clf, hold on, box on;

r_list = linspace(2.5,3.8,1500);

XX = []; YY = [];

%go through r values in for loop
for i = 1:length(r_list)
   N = 0.3;
   r = r_list(i);

   %simulate population
   for t = 1:1000
          N(t+1) = r*N(t)*(1 - N(t));
   end

   %save 200 values in time series
   Y = N(800:end);

   X = ones(size(Y))*r;

   XX = [XX X]; YY = [YY Y];
end

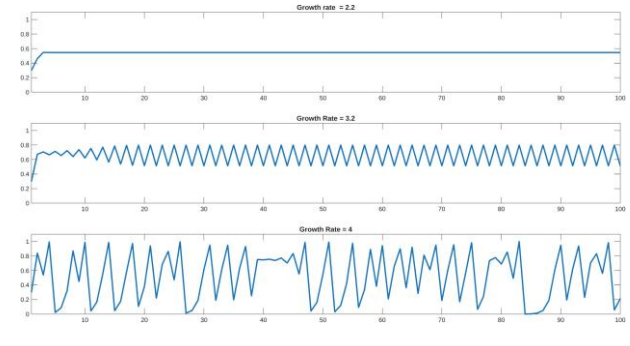
plot(XX, YY, '.', 'MarkerSize', 1, 'Color', 'b');
xlabel('r (Growth Rate Value)');
ylabel('Attractor Set');
title('Bifurcation Diagram');
```

Show logistic-map time series and the bifurcation diagram for growth-rate values spanning 2.6 to 3.8.

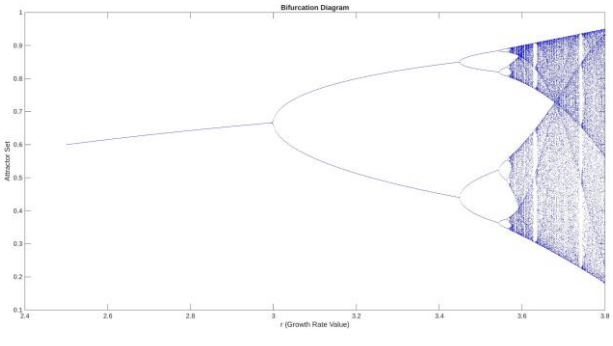
Execution evidence

```
>> task2
>>
```

Output



Task2_Figure_1.png - logistic-map time series



Task2_Figure_2.png - bifurcation diagram

Mandelbrot set and zooms

Render the complete Mandelbrot set and two additional images focused on different regions.

Code - week3/task3.m

Source file: week3/task3.m

```
niter = 30;

%def search area
x0 = -2; x1 = 2;
y0 = -2; y1 = 2;

%def grid
res = 1000;
X = linspace(x0,x1,res);
Y = linspace(y0,y1,res);
[x,y] = meshgrid(X,Y);

c = x + 1i * y;

%init z value
z = zeros(size(c));
k = zeros(size(c));

%repeat apply to map
for ii = 1:niter
   z = z.^2 + c;

   %color grid if still bound
   k(abs(z) > 2 & k == 0) = niter - ii;
end

figure(4), clf
colormap hot
imagesc(X,Y,k);
axis square
set(gca,'xtick',[],'ytick',[])

niter = 200;
G = 6.5e-4;
Center = [-0.7453 0.1127];
x0 = Center(1) - G; x1 = Center(1) + G;
y0 = Center(2) - G; y1 = Center(2) + G;

% Redefine grid with new limits
X = linspace(x0, x1, res);
Y = linspace(y0, y1, res);
[x, y] = meshgrid(X, Y);

c = x + 1i * y;

% Reinitialize z and k for new iteration
z = zeros(size(c));
k = zeros(size(c));

% Repeat the mapping process for the new grid
for ii = 1:niter
   z = z.^2 + c;
   k(abs(z) > 2 & k == 0) = niter - ii;
end
figure(5), clf
colormap hot
imagesc(X, Y, k);
axis square
set(gca, 'xtick', [], 'ytick', []);

%third plot
niter = 200;
G = 60.5e-4;
Center = [-0.7453 0.1127];
x0 = Center(1) - G; x1 = Center(1) + G;
y0 = Center(2) - G; y1 = Center(2) + G;

% Redefine grid with new limits
X = linspace(x0, x1, res);
Y = linspace(y0, y1, res);
[x, y] = meshgrid(X, Y);
```

Code continued - week3/task3.m

Source file: week3/task3.m

```
c = x + 1i * y;

% Reinitialize z and k for new iteration
z = zeros(size(c));
k = zeros(size(c));

% Repeat the mapping process for the new grid
for ii = 1:niter
   z = z.^2 + c;
   k(abs(z) > 2 & k == 0) = niter - ii;
end

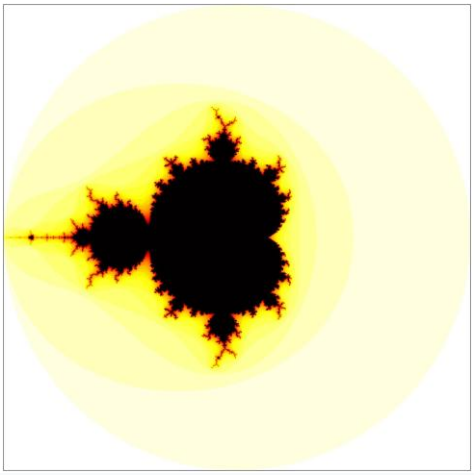
figure(6), clf
colormap hot
imagesc(X, Y, k);
axis square
set(gca, 'xtick', [], 'ytick', []);
```

Render the complete Mandelbrot set and two additional images focused on different regions.

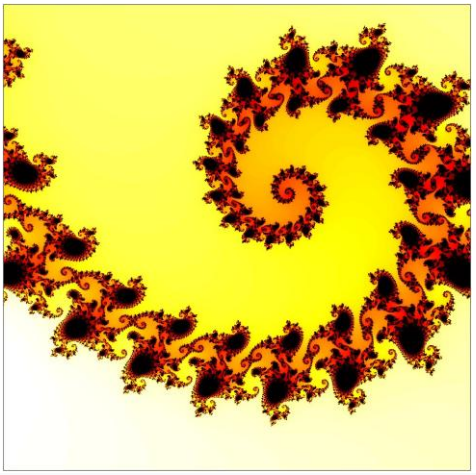
Execution evidence

```
>> task3
>>
```

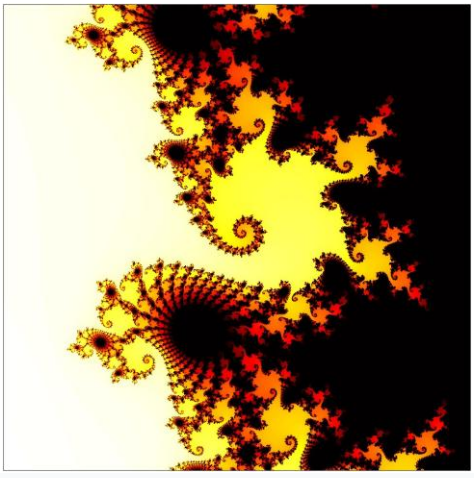
Output



Task3_Figure_1.png - complete Mandelbrot set



Task3_Figure_2.png - detailed region 1



Task3_Figure_3.png - detailed region 2

London Underground population animation

Animate station trip numbers over every time period and produce the final animation as an MPEG-4 file.

Code - week5/londonfixed_fixed.m

Source file: week5/london_animation_fixed.m

```
%% London underground

clear
close all

GRAY10 = [0.1 0.1 0.1];
GRAY40 = [0.4 0.4 0.4];
GRAY80 = [0.8 0.8 0.8];

fig = figure;

smallAx = axes(fig, Position=[0.1 0.7 0.3 0.25], Color=[0.3 0.3 0.3]);
box on
hold on
xlim([0 24])
ylim([0 2])
set(smallAx, xtick=0:2:24, color=GRAY80)
xlabel(smallAx, "Hour", FontSize=12, Color=GRAY80)
ylabel(smallAx, "Trips", FontSize=12, Color=GRAY80)

ax = axes(fig, Position=[0 0 1 1]); hold on;

load London_road_water_data_MXB362.mat Rx* Ry* Thames_river

plot3(ax, Rx1, Ry1, 0*Rx1, Color=GRAY40, LineWidth=1) % small roads
plot3(ax, Rx2, Ry2, 0*Rx2, Color=GRAY40, LineWidth=3) % large roads

cellfun(@(part) ...
    patch(ax, part(:,1), part(:,2), "b", ...
        EdgeColor="none", LineWidth=2, FaceAlpha=0.7, BackFaceLighting="unlit"), ...
    Thames_river ...
);

set(ax, Color=GRAY10)
set(fig, Color=GRAY10)
axis off

view([-13.5, 30])
zlim([0 5000])
xlim([-0.5 0.33])
ylim([51.28 51.70])
[Xc,Yc,Zc] = cylinder(4e-3, 4);

camlight

vid = VideoWriter("LondonUnderground.mp4", "MPEG-4");
vid.FrameRate = 10;
vid.Quality = 100;
open(vid);

load Underground_station_data_MXB362.mat Station_data

% Pre allocating the station columns
surfs = {};
for i = 1:height(Station_data)
    surfs{i} = surf(ax, Xc+Station_data{i,2}, Yc+Station_data{i,3}, Zc .* Station_data{i,4}, ...
        EdgeColor="none", FaceColor=[1 0.6 0.6], FaceAlpha=1.0, SpecularColorReflectance=0.5);
end

TotalPassengers = [0];
for t = 1:96

    for i = 1:height(Station_data)
        surfs{i}.ZData = Zc .* Station_data{i,t+3};
    end

    TotalPassengers = [TotalPassengers 1e-5 .* sum([Station_data{:,t+3}])];
    plot(smallAx, 24 .* [t-1 t]./100, [TotalPassengers(t), TotalPassengers(t+1)], "w", LineWidth=2)

    frame = getframe(fig);
    writeVideo(vid, frame);

end

close(vid)
```

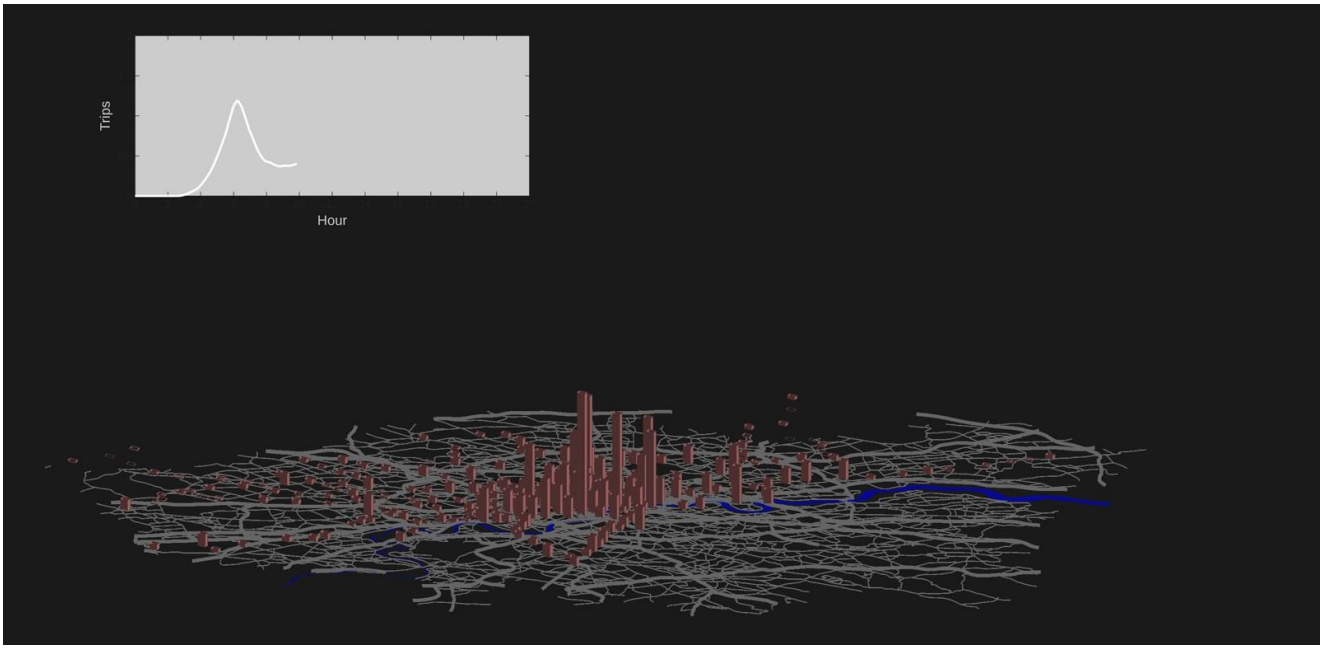
Animate station trip numbers over every time period and produce the final animation as an MPEG-4 file.

Execution evidence

```
>> londonfixed_fixed
>>
es
```

Output

Repository animation: LondonUnderground.mp4.avi. Include the final converted LondonUnderground.mp4 in the submission ZIP.



Representative frame from LondonUnderground.mp4.avi

Pendulum Animation

Solve the pendulum equations of motion, animate the pendulums, and save the result as a GIF.

Code - week5/pendulum2_fixed.m

Source file: week5/pendulum2_fixed.m

```
%% Pendulum

DE2 = @(x,t,g,L) [x(2); -g/L * sin(x(1))];

L0 = 1; % length of first pendulum
n = 16; % num pendulums
R = 51; % decay rate of pendulum length
g = 9.81; % gravity

IC = [pi/6, 0]; % initial angle and time

for i = 1:n

    L = L0 * (R/(R+i-1)).^2;

    [t,x] = ode45(@(t,x) DE2(x,t,g,L), 0:0.05:120, IC);

    angles = x(:,1);

    Y(:,i) = -L .* cos(angles);
    X(:,i) = L .* sin(angles);

end
```

```
h = figure; hold on;
view([-90 -5]); camlight; axis off vis3d
xlim([0 1.2]); ylim([-1.1 1.1]); zlim([-1.1 0]);

% background
patch([1.1 1.1 1.1 1.1], [-1 1 1 -1], [0 0 -1 -1], [0.5 0.5 0.5]);

% colormap
CL = parula(n);
CL = flip(CL, 1);

% preallocate graphics objects
lines = {};
markers = {};
for t = 1:1
    for i = 1:n
        lines{i} = plot3([i i]./n, [0 X(t,i)], [0 Y(t,i)], "-", Color=[0.8 0.8 0.8], MarkerSize=20);
        markers{i} = plot3(i./n, X(t,i), Y(t,i), ".", Color=CL(i,:), MarkerSize=20);
    end
end

% update only Y data and Z data then write to gif

filename = "Pendulum.gif";

if isfile(filename)
    delete(filename)
end

for t = 1:height(X)

    for i = 1:n
        lines{i}.YData = [0 X(t,i)];
        lines{i}.ZData = [0 Y(t,i)];

        markers{i}.YData = X(t,i);
        markers{i}.ZData = Y(t,i);
    end

    drawnow

    frame = getframe(h);
    im = frame2im(frame);
    [imind, cm] = rgb2ind(im, 256);

    if t == 1
        imwrite(imind, cm, filename, 'gif', ...
            'LoopCount', inf, ...
            'DelayTime', 0.02);
    else
        imwrite(imind, cm, filename, 'gif', ...
            'WriteMode', 'append', ...
            'DelayTime', 0.02);
    end

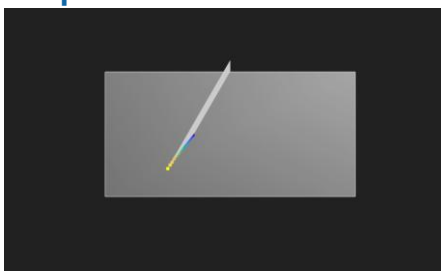
end
```

Solve the pendulum equations of motion, animate the pendulums, and save the result as a GIF.

Execution evidence

```
>> pendulum2_fixed
>>
```

Output



MRI Cylindrical Output

Cut a half cylinder of radius 50 into the MRI volume and render the volume from the top and bottom.

Code - week6_volume_visualisation.R - Task 1

Source file: week6/week6_volume_visualisation.R

```
setwd("A:/MXB362/MXB362Portfolio/week6")

library(R.matlab)

# Read the file directly from the same folder
mat <- readMat("DiCubic.mat")

# Now your previous commands will work
Di <- mat$Di
maxDi <- max(Di)
alpha <- Di / maxDi
C <- Di

composite_z <- function(C, alpha, z_order) {
  Cout <- matrix(0, nrow = dim(C)[1], ncol = dim(C)[2])
  aout <- matrix(0, nrow = dim(C)[1], ncol = dim(C)[2])

  for (i in z_order) {
    Ci <- C[,,i]
    ai <- alpha[,,i]

    Cout <- Cout + (1 - aout) * ai * Ci
    aout <- aout + (1 - aout) * ai
  }

  Cout
}

for (x in 1:256) {
  for (z in 1:54) {
    if ((x - 128)^2 + (z - 54)^2 <= 50^2) {
      alpha[x,,z] <- 0
    }
  }
}

top_render <- composite_z(C, alpha, 1:54)
bottom_render <- composite_z(C, alpha, 54:1)

png("portfolio_outputs/task1_mri_cylindrical_cutout.png",
    width = 1200, height = 600)

par(mfrow = c(1, 2))

image(
  t(apply(top_render, 2, rev)),
  col = gray.colors(256),
  axes = FALSE,
  main = "Top"
)

image(
  t(apply(bottom_render, 2, rev)),
  col = gray.colors(256),
  axes = FALSE,
  main = "Bottom"
)

dev.off()
```

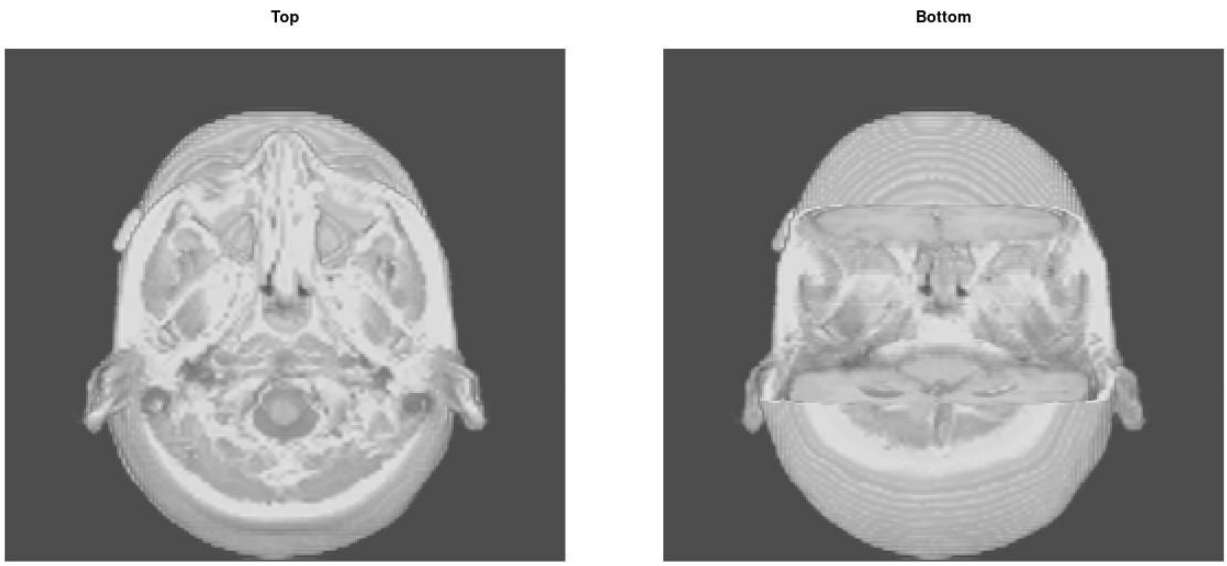
MRI Cylindrical cutout

Cut a half cylinder of radius 50 into the MRI volume and render the volume from the top and bottom.

Execution evidence

```
>
> # Read the file directly from the same folder
> mat <- readMat("DiCubic.mat")
>
> # Now your previous commands will work
> Di <- mat$Di
> maxDi <- max(Di)
> alpha <- Di / maxDi
> C <- Di
>
> composite_z <- function(C, alpha, z_order) {
+   Cout <- matrix(0, nrow = dim(C)[1], ncol = dim(C)[2])
+   aout <- matrix(0, nrow = dim(C)[1], ncol = dim(C)[2])
+
+   for (i in z_order) {
+     Ci <- C[,,i]
+     ai <- alpha[,,i]
+
+     Cout <- Cout + (1 - aout) * ai * Ci
+     aout <- aout + (1 - aout) * ai
+   }
+
+   Cout
+ }
>
> for (x in 1:256) {
+   for (z in 1:54) {
+     if ((x - 128)^2 + (z - 54)^2 <= 50^2) {
+       alpha[x,,z] <- 0
+     }
+   }
+ }
>
> top_render <- composite_z(C, alpha, 1:54)
> bottom_render <- composite_z(C, alpha, 54:1)
>
> png("portfolio_outputs/task1_mri_cylindrical_cutout.png",
+     width = 1200, height = 600)
>
> par(mfrow = c(1, 2))
>
> image(
+   t(apply(top_render, 2, rev)),
+   col = gray.colors(256),
+   axes = FALSE,
+   main = "Top"
+ )
>
> image(
+   t(apply(bottom_render, 2, rev)),
+   col = gray.colors(256),
+   axes = FALSE,
+   main = "Bottom"
+ )
>
> dev.off()
null device
          1
>
```

Output



task1_mri_cylindrical_cutout.png - top and bottom renders

Tooth Slice

Load the CT tooth volume and include a clearly oriented slice image in the portfolio.

Code - week6_volume_visualisation.R - Task 2

Source file: week6/week6_volume_visualisation.R

```
# Task 2 - Tooth slice

mat <- readMat("ctTooth128x128x256.mat")
ctTooth128x128x256 <- mat$ctTooth128x128x256

slice <- ctTooth128x128x256[64,,]
rotated_slice <- t(apply(slice, 2, rev))

png("portfolio_outputs/task2_tooth_slice.png",
    width = 600, height = 900)

image(
  rotated_slice,
  col = gray.colors(256),
  axes = FALSE
)

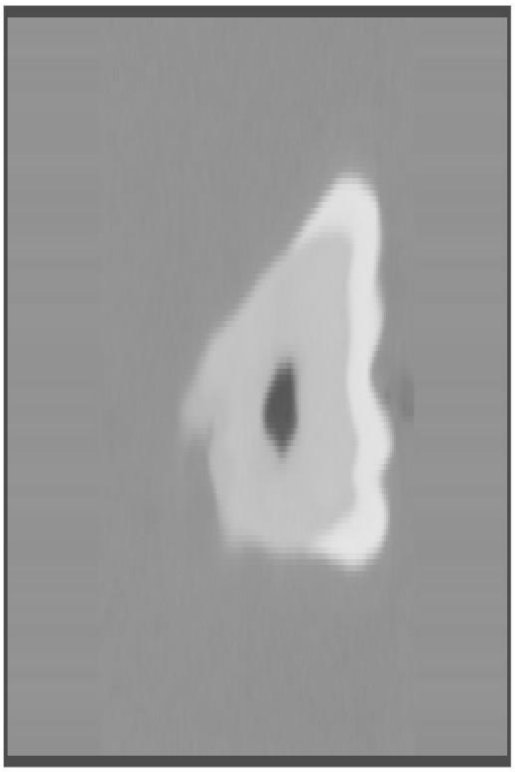
dev.off()
```

Load the CT tooth volume and include a clearly oriented slice image in the portfolio.

Execution evidence

```
> # Task 2 - Tooth slice
>
> mat <- readMat("ctTooth128x128x256.mat")
> ctTooth128x128x256 <- mat$ctTooth128x128x256
>
> slice <- ctTooth128x128x256[64,,]
> rotated_slice <- t(apply(slice, 2, rev))
>
> png("portfolio_outputs/task2_tooth_slice.png",
+     width = 600, height = 900)
>
> image(
+   rotated_slice,
+   col = gray.colors(256),
+   axes = FALSE
+ )
>
> dev.off()
null device
          1
```

Output



task2_tooth_slice.png

Tooth transfer function render

Render the tooth using compositing along the Y axis and a selected linear opacity transfer function.

Code - week6_volume_visualisation.R - Task 3

Source file: week6/week6_volume_visualisation.R

```
# Task 3 - Tooth opacity transfer function

composite_y <- function(C, alpha) {
  Cout <- matrix(0, nrow = dim(C)[1], ncol = dim(C)[3])
  aout <- matrix(0, nrow = dim(C)[1], ncol = dim(C)[3])

  for (jj in 1:dim(C)[2]) {
    Ci <- C[,jj,]
    ai <- alpha[,jj,]

    Cout <- Cout + (1 - aout) * ai * Ci
    aout <- aout + (1 - aout) * ai
  }

  Cout
}

L <- 256
trans_func <- rep(0, L)

trans_func[101:160] <- seq(0, 0.10, length.out = 60)
trans_func[161:220] <- seq(0.10, 0.35, length.out = 60)
trans_func[221:256] <- seq(0.35, 0.80, length.out = 36)

minMRI <- min(ctTooth128x128x256)
maxMRI <- max(ctTooth128x128x256)
lenMRI <- maxMRI - minMRI

alpha <- array(0, dim = dim(ctTooth128x128x256))

for (ii in 1:dim(ctTooth128x128x256)[1]) {
  for (jj in 1:dim(ctTooth128x128x256)[2]) {
    for (kk in 1:dim(ctTooth128x128x256)[3]) {
      volVal <- ctTooth128x128x256[ii,jj,kk]

      opacityIndex <- trunc(
        (volVal - minMRI) / lenMRI * (L - 1)
      ) + 1

      alpha[ii,jj,kk] <- trans_func[opacityIndex]
    }
  }
}

C <- ctTooth128x128x256
tooth_render <- composite_y(C, alpha)
rotated_render <- t(apply(tooth_render, 2, rev))

png("portfolio_outputs/task3_transfer_function.png",
    width = 800, height = 500)

plot(
  trans_func,
  type = "l",
  xlab = "Opacity index",
  ylab = "Opacity",
  main = "Tooth Opacity Transfer Function"
)

dev.off()

png("portfolio_outputs/task3_tooth_render.png",
    width = 600, height = 900)

image(
  rotated_render,
  col = gray.colors(256),
  axes = FALSE
)

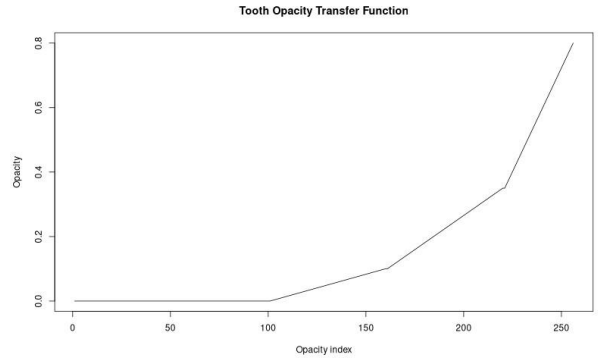
dev.off()
```

Render the tooth using compositing along the Y axis and a selected linear opacity transfer function.

Execution evidence

```
> # Task 3 - Tooth opacity transfer function
>
> composite_y <- function(C, alpha) {
+   Cout <- matrix(0, nrow = dim(C)[1], ncol = dim(C)[3])
+   aout <- matrix(0, nrow = dim(C)[1], ncol = dim(C)[3])
+
+   for (jj in 1:dim(C)[2]) {
+     Ci <- C[,jj,]
+     ai <- alpha[,jj,]
+
+     Cout <- Cout + (1 - aout) * ai * Ci
+     aout <- aout + (1 - aout) * ai
+   }
+
+   Cout
+ }
>
> L <- 256
> trans_func <- rep(0, L)
>
> trans_func[101:160] <- seq(0, 0.10, length.out = 60)
> trans_func[161:220] <- seq(0.10, 0.35, length.out = 60)
> trans_func[221:256] <- seq(0.35, 0.80, length.out = 36)
>
> minMRI <- min(ctTooth128x128x256)
> maxMRI <- max(ctTooth128x128x256)
> lenMRI <- maxMRI - minMRI
>
> alpha <- array(0, dim = dim(ctTooth128x128x256))
>
> for (ii in 1:dim(ctTooth128x128x256)[1]) {
+   for (jj in 1:dim(ctTooth128x128x256)[2]) {
+     for (kk in 1:dim(ctTooth128x128x256)[3]) {
+       volVal <- ctTooth128x128x256[ii,jj,kk]
+
+       opacityIndex <- trunc(
+         (volVal - minMRI) / lenMRI * (L - 1)
+       ) + 1
+
+       alpha[ii,jj,kk] <- trans_func[opacityIndex]
+     }
+   }
+ }
>
> C <- ctTooth128x128x256
> tooth_render <- composite_y(C, alpha)
> rotated_render <- t(apply(tooth_render, 2, rev))
>
> png("portfolio_outputs/task3_transfer_function.png",
+     width = 800, height = 500)
>
> plot(
+   trans_func,
+   type = "l",
+   xlab = "Opacity index",
+   ylab = "Opacity",
+   main = "Tooth Opacity Transfer Function"
+ )
>
> dev.off()
null device
          1
>
> png("portfolio_outputs/task3_tooth_render.png",
+     width = 600, height = 900)
>
> image(
+   rotated_render,
+   col = gray.colors(256),
+   axes = FALSE
+ )
>
> dev.off()
null device
          1
>
```

Output



task3_transfer_function.png - selected opacity mapping



task3_tooth_render.png - final volume render